OCR Word Search Solver

-

EPITA S3

Rapport de soutenance 1

Octobre 2026

Équipe : Sudocrew
Gustave SCHWIEN
Guillaume KLEIN
Antoine MUH
Antoine LAPP

Sommaire

Rôle et mission de chaque membre....3
Présentations individuelles :....3
Gustave....3
Guillaume....3
Antoine Lapp....3
Antoine Muh....4
Objectifs, planification et état d'avancement....4
Objectifs....4
Planification des tâches....4
Avancement attendu et réel....5
Gustave....5
Guillaume....6
Antoine Lapp....6
Antoine Muh....6
Choix, justifications et difficultés par objectif....6
Interface et traitement de l'image (Gustave)....6
Architecture de l'interface....6
Fonctions de prétraitement....7
Choix et justifications....8
Difficultés....9
Segmentation et analyse de l'image (Antoine LAPP)....9
Découpage de l'image en macro-régions....9
Classification des régions....10
Récupération des caractères....12
Réseau de neurones (Guillaume)....13
Fonctionnement du réseau implémenté....13
Choix et justifications....14
Difficultés....15
Solver (Antoine Muh)....16
Algorithme....16
Choix et justifications....16
Tests....17
Difficultés....17
Tâches futures pour la suite du projet....17
Gustave....17
Guillaume....17
Antoine Lapp....18
Antoine Muh....18
Bibliographie....18
Commune....18
Gustave....18
Guillaume....19
Antoine Lapp....19
Antoine MUH....19

Rôle et mission de chaque membre

Le projet est découpé en quatre parties, une par membre, pour que chacune puisse avancer en parallèle. Les formats échangés entre les parties (images, lettres découpées, grille de lettres) ont été fixés au début du projet, pour que chacun puisse coder et tester sa partie sans attendre les autres.

Membre	**Partie**
Gustave	Interface et traitement de l'image
Guillaume	Réseau de neurones
Antoine Lapp	Segmentation, classification des zones
Antoine Muh	Solver, interface graphique et intégration

Présentations individuelles :

Gustave

Je suis responsable de l'interface graphique de l'application ainsi que de l'implémentation des algorithmes de prétraitement de l'image. Pour cette soutenance, je me suis occupé de développer une interface graphique accessible et intuitive comprenant une zone de prévisualisation de l'image de la grille et un panneau de contrôle où se trouvent un curseur faisant tourner l'image, un bouton de sauvegarde de l'image terminée (pas encore actif) et le bouton de lancement de la résolution de la grille, pour l'instant limitée à sa segmentation. J'ai également implémenté la fonction de conversion de l'image en niveaux de gris et celle de sa binarisation. Pour la soutenance finale, je vais ajouter un algorithme de rotation automatique pour aligner la grille et permettre sa bonne segmentation, et je vais peaufiner l'interface en y ajoutant les fonctionnalités restantes imposées par le cahier des charges.

Guillaume

Je suis responsable du réseau de neurones chargé de reconnaître les lettres de la grille et de la liste de mots. Pour cette soutenance, ma mission est la preuve de concept demandée par le sujet : un petit réseau capable d'apprendre la fonction logique $\bar{A}.\bar{B} + A.B$. J'ai écrit ce code pour qu'il serve tel quel au réseau final, et pas seulement à cette démonstration.

Pour la soutenance finale, ma mission s'étend à : la construction du jeu de données de lettres ; l'entraînement du réseau sur les images 28 × 28 produites par la segmentation; la reconnaissance des lettres.

Antoine Lapp

Mon rôle dans ce projet sur cette période a été d'analyser l'image prétraitée afin de détecter les différentes régions utiles pour la résolution du jeu : la grille de mots mêlés et la liste de mots à

trouver. Pour cela, il a fallu segmenter l'image en plusieurs régions et classifier chacune d'entre elles afin d'ignorer celles qui sont inutiles et d'utiliser correctement celles qui sont nécessaires à la résolution du jeu. Afin de faire cela, j'ai dû m'appuyer sur des données statistiques de chaque zone. Ensuite, j'ai créer plusieurs boîtes englobant chaque caractère des régions utiles et stocker ces boîtes de façon organisée pour qu'on sache où se trouve chaque caractère après leur passage dans le réseau de neurones. Enfin, je me suis chargé de redimensionner ces boîtes afin de les faire correspondre à une matrice de dimensions 28 par 28 pixels ce qui est nécessaire au réseau de neurones.

Antoine Muh

Mon rôle couvre la fin de la chaîne de traitement. Pour cette soutenance, ma mission a été le programme en ligne de commande solver, qui cherche un mot dans une grille de lettres. Pour la soutenance finale, elle s'étend à la reconstruction de la grille et de la liste de mots à partir des lettres reconnues, à l'affichage et à la sauvegarde de la grille résolue, et à l'interface graphique. Je me suis aussi chargé de la cohérence du dépôt GIT et de la mise en commun du Makefile.

Objectifs, planification et état d'avancement

Objectifs

Le logiciel doit résoudre une grille de mots cachés à partir d'une image : prétraiter l'image, repérer la grille et la liste de mots, reconnaître chaque lettre avec un réseau de neurones, reconstruire la grille, puis y chercher les mots et afficher le résultat. Pour cette première soutenance, les éléments suivants sont terminés :

1. Chargement d'une image et suppression des couleurs.
2. Rotation manuelle de l'image.
3. Détection de la position de la grille, de la liste de mots, des lettres de la grille, des mots de la liste et des lettres de ces mots.
4. Découpage de l'image, avec la sauvegarde de chaque lettre sous forme d'image.
5. Algorithme de résolution dans le programme en ligne de commande solver.
6. Preuve de concept du réseau de neurones sur la fonction logique $A.B + \bar{A}.\bar{B}$.

Planification des tâches

La semaine du 14 septembre a servi à la mise en place commune : constitution du groupe, choix de SDL2 et SDL2_image pour le traitement des images et pour l'interface. Le planning concentre ensuite la phase 1 sur trois semaines chargées, du 21 septembre au 11 octobre :

Semaine	**Gustave**	**Guillaume**	**Antoine Lapp**	**Antoine Muh**
21 - 27 septembre	Chargement, niveaux de gris, binarisation	Preuve de concept $A.B + \bar{A}.\bar{B}$	Projections d'histogrammes, détection de la grille	Solver : 8 directions, format de sortie exact
28 septembre - 4 octobre	Rotation manuelle avec curseur de contrôle	Architecture du réseau complet	Détection des cases puis des lettres, découpage	Tests du solver

5 - 11 octobre	Interface : boutons principaux et attribution de leurs fonctions	Premiers entraînements, mesure du taux de reconnaissance	Détection de la liste, des mots et de leurs lettres, export des lettres	Script de démonstration, plan de soutenance et commencement du rapport

Les semaines du 12 au 25 octobre ont été laissées sans tâche pour rattraper les potentiels retards. La semaine du 26 octobre est réservée à l'assemblage : gel du code le 28 octobre, rendu du dépôt le 30 octobre à 20h.

Avancement attendu et réel

Le tableau reprend les éléments attendus pour cette soutenance et leur état au moment du rendu.

Élément attendu	**Responsable**	**Prévu**	**État**
Chargement de l'image, suppression des couleurs	Gustave	27 septembre	Terminé
Rotation manuelle	Gustave	4 octobre	Terminé
Boutons principaux	Gustave	11 octobre	Terminé
Détection de la grille	Antoine Lapp	27 septembre	Terminé
Détection des cases et des lettres, découpage	Antoine Lapp	4 octobre	Terminé
Détection de la liste, des mots et de leurs lettres, export des lettres	Antoine Lapp	11 octobre	Terminé
Preuve de concept du réseau de neurones	Guillaume	27 septembre	Terminé
Programme solver	Antoine Muh	27 septembre	Terminé

Chaque membre détaille ci-dessous l'avancement de sa partie.

Gustave

Malgré des difficultés à respecter les délais en début d’année, dûes à un manque de temps libre pour des raisons personnelles, j’ai réussi à réaliser l’ensemble de mes tâches à temps pour la soutenance. L’interface de l’application n’est pas entièrement terminée mais ses composants principaux sont là : le chargement et l’affichage de l’image binarisée à l’aide des fonctions de prétraitement (niveaux de gris et puis noir et blanc avec seuil adaptatif), le curseur permettant de faire tourner l’image manuellement, le bouton d’enregistrement de l’image après résolution de la grille (pas encore utilisé car on ne peut pas encore résoudre de grille), et le bouton de lancement de l’analyse qui a été relié au programme de détection et de découpage de la grille, première étape de l’analyse.

Guillaume

La preuve de concept du réseau de neurones devait être terminée le 27 septembre. J'ai mis en place les étapes une par une, chacune dans sa propre branche. La structure du réseau et l'initialisation des poids ont été faites fin septembre, puis la propagation avant, le softmax, l'entropie croisée et la rétropropagation début octobre, et enfin la descente de gradient le 5 octobre. L'apprentissage a fonctionné le 13 octobre, avec environ deux semaines de retard, dues surtout au débogage de l'entraînement. Les semaines tampons ont servi à rattraper ce retard, et le réseau a été intégré au dépôt commun le 24 octobre. Il apprend la fonction $\bar{A}.\bar{B} + A.B$ et répond correctement sur les 4 cas, avec une perte moyenne de 0,009 après 2 000 époques. J'ai aussi pris de l'avance sur la soutenance finale en ajoutant la sauvegarde et le chargement des poids dans un fichier : un réseau rechargé donne exactement les mêmes résultats que le réseau d'origine.

Antoine Lapp

Les dates initialement prévues pour les différentes tâches liées à mon rôle dans ce projet pour cette première soutenance étaient mal estimées relativement à la nature de celles-ci : le découpage, redimensionnement et export en matrice des caractères individuels ne m'a pris qu'une semaine tandis que la détection de macro zones dans l'image m'a pris plusieurs semaines à cause des difficultés que j'ai rencontrées pour arrêter correctement la récursivité de mon algorithme de segmentation de l'image. De plus, à cause d'une contrainte personnelle qui m'a empêché de travailler sur ce projet durant une grande partie des vacances, j'ai dû anticiper afin de contrebalancer ce manque de travail sur cette période et prendre de l'avance dans mon travail. J'estime que ceci a été plutôt bien réussi car mon module d'analyse d'image était complètement fonctionnel sur des images simples avant la période de vacances ce qui a permis de ne pas retarder le groupe.

Antoine Muh

Le solver devait être terminé la semaine du 27 septembre. Il a été terminé début octobre, avec environ une semaine de retard suite à des memory leaks. Il a enfin été intégré au dépôt commun mi-octobre. Les semaines tampons ont servi à rattraper ce retard. Le programme solver a été testé et validé sur les 8 exemples du sujet. Le groupe dispose à présent du solver terminé et fonctionnel pour toute la durée du projet. J'ai ensuite profité de cette charge de travail moins lourde sur la dernière semaine pour préparer les documents nécessaires à cette soutenance tel que le plan de celle-ci ou la perfection de la démo.

Choix, justifications et difficultés par objectif

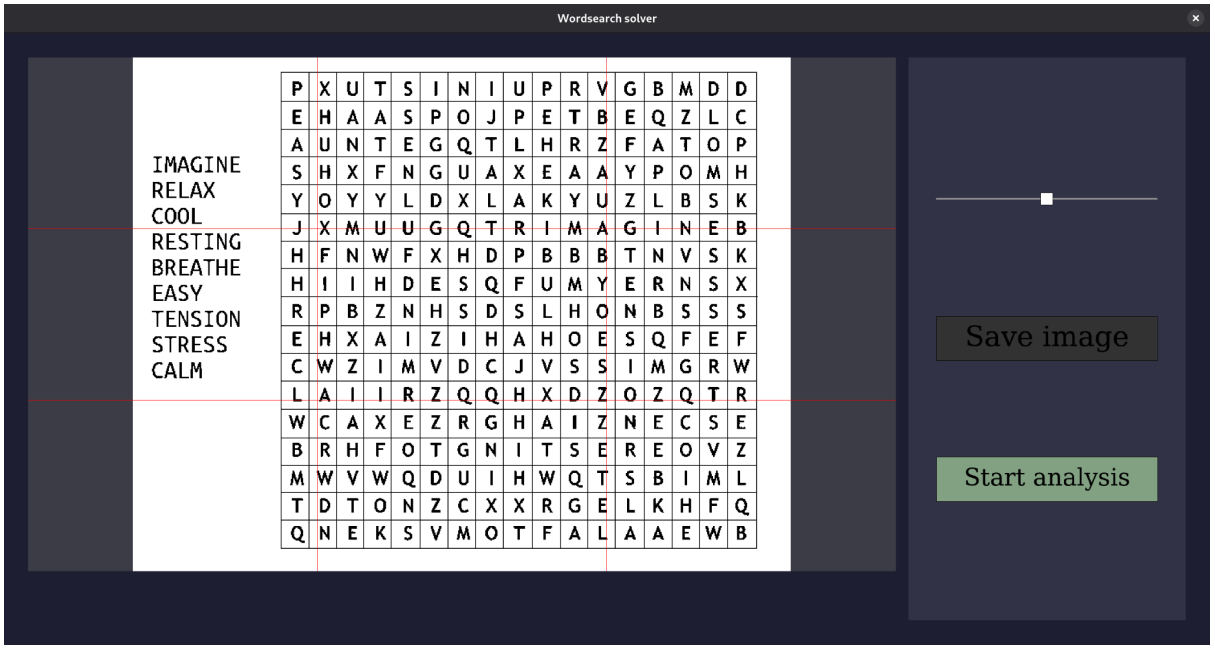
Chaque membre présente les choix technologiques et algorithmiques de sa partie, leur justification et les difficultés rencontrées.

Interface et traitement de l'image (Gustave)

Architecture de l'interface

Avant de commencer à programmer l'interface, j'ai dû commencer par réfléchir à une architecture propre, intuitive et facile d'utilisation, afin que l'utilisateur puisse la prendre en main rapidement. Le cahier des charges nous imposait le chargement de l'image dans l'interface, et la possibilité de la faire tourner manuellement afin que le réseau de neurones travaille sur une image alignée

correctement. J'ai donc décidé de dédier la majorité de la fenêtre à une zone rectangulaire de prévisualisation pour afficher et tourner l'image avant analyse, avec l'idée d'y afficher plus tard l'image de la grille résolue, avec des boutons permettant de passer de l'une à l'autre.
L'interface avait aussi besoin d'un panneau de contrôle contenant les boutons principaux de l'application. J'ai placé ce panneau dans la partie droite de la fenêtre.



Capture de la fenêtre actuelle de l'application, avec la zone de prévisualisation et le panneau de contrôle.

Fonctions de prétraitement

Pour le prétraitement, j'ai réalisé 3 fonctions. La première convertit chaque pixel en un niveau de gris en utilisant la formule qui nous avait été fournie en TP de programmation :

$$gray = 0.299 * r + 0.587 * g + 0.114 * b$$

La deuxième devait être un outil de débruitage utilisant un algorithme de filtre médian, mais j'ai fini par la supprimer en réalisant qu'un filtre n'était pas adapté aux images créées de toutes pièces comme celles des grilles de mots mêlés : le filtre ne faisait que flouter les lettres, ce qui rend plus difficile la reconnaissance de caractères.
La troisième est la fonction de binarisation de l'image, qui convertit chaque pixel gris soit en noir pur, soit en blanc pur. Pour ce faire, au lieu de simplement vérifier si le pixel est plus proche du noir ou du blanc, on utilise un algorithme qui prend en compte les pixels environnants, et qui calcule leur luminosité moyenne. Si le pixel actuel est inférieur de t% à la moyenne locale (t étant un seuil adaptatif choisi), alors le pixel devient noir. Sinon, il est blanc. J'ai placé notre seuil t à 10, valeur pour laquelle la binarisation était la mieux réalisée.

Choix et justifications

Choix	**Justification**
Utilisation de SDL pour l'interface et SDL_image pour le traitement d'image au lieu de GTK	J'ai choisi d'utiliser mes connaissances acquises en cours pour développer l'interface au lieu de réapprendre à utiliser une toute autre bibliothèque.
Fenêtre à taille variable et zones de rendu dynamiques	Bien que cela demande de recalculer la position des différents éléments à chaque mise à jour, la possibilité de redimensionner la fenêtre rend tout de suite l'application beaucoup plus propre, et lui permet de s'adapter à toute taille et résolution d'écran.
Chemin d'accès de l'image en argument de la commande d'exécution de l'application	SDL ne possédant pas de boîte de dialogue système facile d'utilisation, il n'était pas possible de charger d'image directement dans l'application.
Affichage de l'image sans étirement	L'application maintient le ratio d'aspect original de l'image chargée pour éviter le comportement par défaut de SDL qui est d'étirer ou de déformer l'image pour lui faire prendre tout l'espace de la zone de prévisualisation.
Utilisation d'un curseur allant de gauche à droite pour la rotation manuelle	Le curseur est la manière la plus intuitive de faire tourner l'image.
Création d'une structure Button pour les boutons de l'interface	SDL ne possédant pas de structure interne de bouton, j'ai donc créé la mienne.
Positionnement des boutons sur 25, 50 et 75% du panneau latéral	Cet agencement garantit l'équidistance des boutons et leur centrage parfait.
Rendu du texte des boutons en images	La bibliothèque SDL_ttf étant absente des machines de l'école, j'ai converti mon texte en image transparente que j'affiche directement sur les boutons. Ce procédé me permet d'ailleurs de choisir facilement une différente police d'écriture.
Ajout d'un quadrillage sur la zone de prévisualisation	Ce quadrillage permet à l'utilisateur d'être plus précis en vérifiant s'il a correctement aligné l'image avec le curseur.
Suppression de la fonction de débruitage	Les images traitées étant des images de mots mêlés créées de toutes pièces, il n'était pas nécessaire de leur appliquer un débruitage car l'algorithme se contentait de flouter les lettres. La binarisation est suffisante.

Difficultés

- **J'ai mis du temps à choisir entre GTK et SDL pour la réalisation de l'interface.** En me renseignant, GTK me semblait plus adapté au projet alors que nous apprenions SDL en cours. Finalement, j'ai choisi la bibliothèque que je connaissais.
- **Je ne trouvais pas de solution pour charger l'image depuis l'application.** J'ai fait beaucoup de recherches sur les moyens de charger un fichier depuis l'explorateur en C, en vain. L'image est finalement chargée lors du démarrage du programme.
- **J'ai eu du mal à déterminer les bonnes formules pour calculer les positions des éléments de l'interface,** mises à jour à chaque redimensionnement de la fenêtre. Je ne pouvais pas leur associer une valeur fixe et les formules étaient parfois complexes.

Segmentation et analyse de l'image (Antoine LAPP)

L'image prétraitée a été transformée en une matrice binaire : elle contient des 0 (pixels noirs) ou des 255 (pixels blancs).

Découpage de l'image en macro-régions

Pour analyser l'image, on la découpe en régions. Certaines sont utiles à la résolution du problème (la grille et la liste de mots), d'autres non et doivent être ignorées.

Algorithme du XY-cut :

J'ai implémenté l'algorithme du XY-cut récursif, qui est notamment utilisé pour découper des documents écrits en paragraphes, titres et images.

On commence par compter le nombre de pixels noirs de chaque ligne de pixels de la région. Les lignes qui n'en contiennent aucune forment des bandes vides, qui séparent des blocs de contenu : on découpe donc la région en bandes horizontales non vides. Ensuite, pour chacune de ces bandes, on recommence en comptant cette fois les pixels noirs de chaque colonne, ce qui la découpe en bandes verticales. On continue ainsi récursivement, en alternant l'axe de découpe à chaque niveau.

Conditions d'arrêt :

Si on descend trop dans les appels récursifs, des éléments qui devraient appartenir à une même région (par exemple deux mots de la liste) se retrouvent séparés dans des zones distinctes. Si on ne descend pas assez, des zones distinctes restent fusionnées et la récupération des informations qu'elles contiennent est bien plus compliquée.

La récursion s'arrête sur une région dans trois cas : elle ne contient aucun pixel noir ; la découpe ne la divise dans aucune des deux directions (on essaie l'autre axe avant d'abandonner) ; elle est trop petite (moins de 1 % de la taille de l'image).

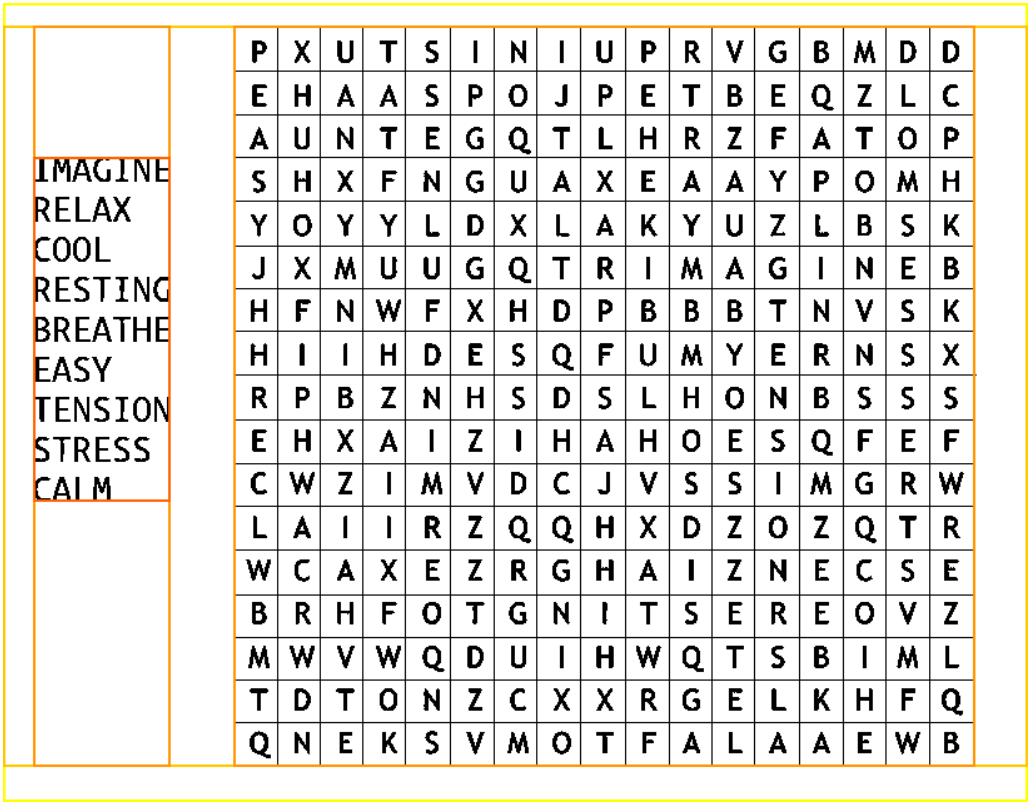
Le paramètre le plus délicat, qui m'a donné beaucoup de difficultés, est la taille à partir de laquelle une bande vide sépare deux régions. J'ai d'abord essayé une valeur fixe en pixels, mais elle dépend de la résolution de l'image, donc elle n'était pas viable. J'ai ensuite essayé un pourcentage de la taille de la zone découpée, sans plus de succès, car ce pourcentage varie selon l'image et selon le niveau de récursion. J'ai aussi essayé de limiter le nombre de sous-régions trouvées par découpe, mais ce critère s'est montré peu fiable et je l'ai abandonné.

Il fallait une valeur directement liée à la taille du texte. J'ai donc créé une fonction qui calcule la hauteur médiane des composantes connexes de l'image (une composante connexe est un ensemble de pixels noirs qui se touchent ; dans la plupart des cas, c'est une lettre). Une bande vide sépare deux régions si sa taille dépasse 1,5 fois cette hauteur. Cette valeur donne des résultats satisfaisants sur des images de résolutions et de tailles de texte différentes : sur nos images de test, tout facteur entre 1,0 et 1,9 convenait.

Enregistrement des zones sous forme d'arbre :

Les régions trouvées à chaque découpe sont stockées dans un arbre. La racine est l'image entière. Les zones de la première découpe sont ses fils, puis chacune est à son tour découpée selon l'autre axe et ses sous-régions deviennent ses propres fils, et ainsi de suite. Chaque région enregistre ses coordonnées, son niveau dans l'arbre et son type. Les macro-régions à classifier sont les feuilles de l'arbre.

On peut voir dans cette image de debug ci dessous les différentes zones englobant des plus petites. On retrouve bien la structure d'arbre et la récursion s'arrête bien à l'échelle des macro zones. L'image tout entière est représenté par la zone jaune, pyuis on effectue une coupure sur l'axe vertical, on élimine alors les espaces inutiles en haut et en bas et dans cette nouvelle zone on effectue une coupe horizontal qui permet de distinguer 2 nouvelles zones et on alterne les coupures ainsi de suite jusqu'à ce qu'on ai obtenu les macro zones :



Classification des régions

Pour classifier les feuilles, on calcule des statistiques sur chacune, regroupées dans une structure : le ratio largeur/hauteur de sa boîte, sa densité en pixels noirs et sa régularité dans les deux directions.

La régularité mesure si les éléments d'une région ont tous la même taille. Dans une direction donnée, on compte les pixels noirs de chaque ligne (ou colonne) ; les suites de lignes (ou colonnes) non vides forment des blocs, par exemple une ligne de texte ou une colonne de lettres. On calcule la taille de chaque bloc, puis le coefficient de variation : l'écart-type de ces tailles divisé par leur moyenne. Il vaut 0 quand tous les blocs ont la même taille et augmente quand ils diffèrent. Comme il n'a pas d'unité, il ne dépend pas de la résolution de l'image, contrairement à la variance.

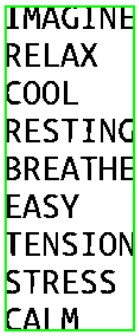
Dans une grille, les lignes de lettres ont toutes la même hauteur et les colonnes la même largeur : la régularité est faible dans les deux directions. Dans une liste de mots, les lignes ont la même hauteur (régularité faible) mais les mots ont des longueurs différentes (régularité élevée sur les colonnes : entre 0,5 et 0,65 sur nos images). Quand une région n'a qu'un seul bloc dans une direction, ce qui arrive avec une grille entourée de bordures continues, la régularité vaut 0 par convention.

Les règles de classification sont les suivantes :

- Si le ratio est très éloigné de 1, la région est une ligne ou une bande fine : elle est inutile.
- Si la densité est quasi nulle (bruit) ou quasi totale (aplat noir), la région est inutile.
- Si le ratio est proche de 1 et si la régularité est inférieure à 0,3 dans les deux directions, la région est la grille. Comme il n'y a qu'une grille par image, on ne garde que la meilleure candidate.

Pour l'instant, je me suis concentré sur des images simples de niveau 1, pour obtenir un résultat qui fonctionne totalement avant de traiter des images plus complexes. J'ai donc considéré que toute zone qui n'est ni inutile ni la grille est une liste de mots. C'est cette partie de l'analyse que je vais améliorer par la suite pour gérer les titres, les consignes et les illustrations.

On obtient donc des régions correspondant aux plus petites zones de l'image précédente (qui correspondent donc aux feuilles de l'arbre). Et grâce au données statistiques on a pu classer les régions. Leur type est représentés ci dessous pas ce code couleur : vert - liste de mots et bleu - grille de lettres



P	X	U	T	S	I	N	I	U	P	R	V	G	B	M	D	D
E	H	A	A	S	P	O	J	P	E	T	B	E	Q	Z	L	C
A	U	N	T	E	G	Q	T	L	H	R	Z	F	A	T	O	P
S	H	X	F	N	G	U	A	X	E	A	A	Y	P	O	M	H
Y	O	Y	Y	L	D	X	L	A	K	Y	U	Z	L	B	S	K
J	X	M	U	U	G	Q	T	R	I	M	A	G	I	N	E	B
H	F	N	W	F	X	H	D	P	B	B	B	T	N	V	S	K
H	I	I	H	D	E	S	Q	F	U	M	Y	E	R	N	S	X
R	P	B	Z	N	H	S	D	S	L	H	O	N	B	S	S	S
E	H	X	A	I	Z	I	H	A	H	O	E	S	Q	F	E	F
C	W	Z	I	M	V	D	C	J	V	S	S	I	M	G	R	W
L	A	I	I	R	Z	Q	Q	H	X	D	Z	O	Z	Q	T	R
W	C	A	X	E	Z	R	G	H	A	I	Z	N	E	C	S	E
B	R	H	F	O	T	G	N	I	T	S	E	R	E	O	V	Z
M	W	V	W	Q	D	U	I	H	W	Q	T	S	B	I	M	L
T	D	T	O	N	Z	C	X	X	R	G	E	L	K	H	F	Q
Q	N	E	K	S	V	M	O	T	F	A	L	A	A	E	W	B

Récupération des caractères

Création des boîtes englobantes :

Dans la région de la grille, on détecte les composantes connexes et on crée une boîte englobante autour de chacune. Deux types de boîtes sont supprimés : celles de plus de 2 fois la hauteur médiane des caractères (la bordure de la grille, qui forme une seule grande composante) et celles de moins de la moitié de cette hauteur (bruit). Comme la binarisation peut couper une lettre en deux morceaux, on fusionne aussi les boîtes très proches l'une de l'autre quand la boîte obtenue garde la taille d'une lettre.

Pour la ou les listes de mots, on fait globalement la même chose, en ajoutant une fonction qui divise les boîtes qui englobent plusieurs lettres collées (c'est impossible dans la grille, où chaque lettre est dans sa case). La hauteur médiane est calculée sur la zone de la liste elle-même, car sa police peut être plus petite que celle de la grille.

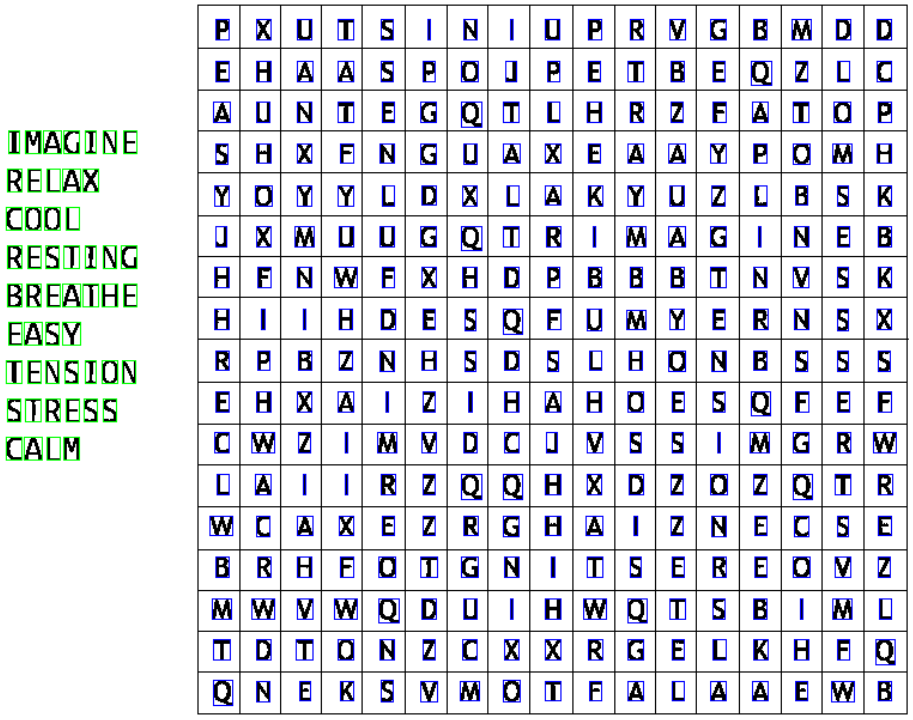
Stockage des boîtes :

Les boîtes sont enregistrées dans un tableau dynamique, avec le nombre de boîtes et la capacité du tableau, qui est doublée quand il est plein. On a choisi un tableau plutôt qu'une liste chaînée pour accéder directement à une lettre par son indice.

Ce tableau est trié dans l'ordre de lecture. On trie d'abord les boîtes selon la position verticale de leur centre, puis on les regroupe en lignes (une nouvelle ligne commence quand le centre d'une boîte s'écarte de plus d'une demi-hauteur de lettre de celui de la ligne en cours), et enfin on trie chaque ligne de gauche à droite. Un second tableau, row_start, donne pour chaque ligne l'indice de sa première boîte dans le grand tableau.

Par exemple, pour une liste de mots IMAGINE, RELAX, COOL, le tableau contient les 16 lettres à la suite, et row_start vaut [0, 7, 12]. Le mot d'indice 1 (RELAX) s'étend donc des indices 7 à 11, et la lettre d'indice j du mot d'indice i se trouve à l'indice row_start[i] + j. Chaque ligne est donc un mot, et on garde ainsi l'appartenance de chaque lettre à son mot et sa position dans l'image. La grille utilise la même structure, avec des lignes toutes de même longueur, ce que l'on vérifie pour détecter les lettres manquantes ou coupées.

Comme on peut le voir dans l'image ci dessous, on peut donc tracer la boîte englobante de chaque caractère dans les zones utiles, la bordure de la grille est bien ignorée :



Redimensionnement et centrage :
On possède donc la boîte englobante de chaque caractère, et on la transforme en matrice de 28 par 28 pixels, format demandé par Guillaume pour le réseau de neurones. On redimensionne le caractère en conservant son ratio, de façon que sa dimension la plus longue occupe la taille de contenu choisie, puis on le centre dans la matrice, complétée par des pixels blancs. Chaque pixel de la matrice vaut la proportion d'encre dans la zone de l'image qu'il représente, ce qui donne des niveaux de gris.

On pourra ensuite appeler le réseau de neurones sur chacune de ces matrices.

Réseau de neurones (Guillaume)

Fonctionnement du réseau implémenté

Le réseau est un perceptron multicouche à une couche cachée (figure 1 en annexe). Pour la preuve de concept, il comporte 2 entrées (A et B), 4 neurones cachés avec la fonction d'activation sigmoïde, et 2 neurones de sortie suivis d'un softmax. Cela représente 22 paramètres : une matrice de poids W de taille 4 × 2 et 4 biais b pour la couche cachée, une matrice V de taille 2 × 4 et 2 biais c pour la sortie. La sortie k donne la probabilité que le résultat de la fonction soit k ; la réponse du réseau est l'indice de la sortie la plus forte.

Propagation avant. Chaque neurone caché calcule $h_i = \sigma(b_i + \Sigma_\square W_{i\square} x_\square)$, avec $\sigma(x) = 1 / (1 + e^{-x})$. Chaque sortie calcule $z_\square = c_\square + \Sigma_i V_{\square i} h_i$. Le softmax transforme ensuite ces valeurs en probabilités $y_\square = e^{z_\square} / \Sigma_\square e^{z_\square}$, dont la somme vaut 1. Le maximum des $z_\square$ est soustrait avant le calcul des exponentielles : le résultat est identique, mais on évite les dépassements de capacité.

Mesure de l'erreur. L'erreur est mesurée par l'entropie croisée $L = -\Sigma_\square t_\square \log(y_\square)$, où t est la sortie attendue sous forme de vecteur one-hot. Par exemple, t = (0, 1) quand le résultat attendu est 1.

Rétropropagation. Avec un softmax suivi de l'entropie croisée, l'erreur de la sortie k se simplifie en $\delta_k = y_k - t_k$. On en déduit :

- le gradient d'un poids de sortie : $\delta_k \times h_i$;
- l'erreur d'un neurone caché : $\delta_i = (\Sigma_k \delta_k V_{ki}) \times h_i (1 - h_i)$, où $h_i (1 - h_i)$ est la dérivée de la sigmoïde ;
- le gradient d'un poids caché : $\delta_i \times x_k$.

Le gradient de chaque biais est égal à l'erreur du neurone correspondant.

Apprentissage. Les poids sont mis à jour après chaque exemple (descente de gradient stochastique) : $w \leftarrow w - \eta \times \partial L/\partial w$. Le taux d'apprentissage η vaut 0,1, et l'apprentissage dure 2 000 époques sur les 4 combinaisons de A et B.

Résultats. L'évolution de la perte moyenne est la suivante :

Époque	**Perte moyenne**
1	0,747
500	0,567
1 000	0,055
2 000	0,009

Après l'apprentissage, le programme affiche :

1.0 == 0.0 = 0\n 0.0 == 0.0 = 1\n 0.0 == 1.0 = 0\n 1.0 == 1.0 = 1

La bonne réponse a une probabilité d'environ 0,99 dans chaque cas. Sur 8 initialisations aléatoires différentes, le réseau donne les 4 bonnes réponses à chaque fois, avec une perte finale comprise entre 0,007 et 0,012.

Sauvegarde. Le réseau est enregistré dans un fichier texte qui contient, dans l'ordre :

1. un en-tête versionné (NN 1) ;
2. les tailles des trois couches ;
3. les poids et les biais de chaque couche, écrits avec 17 chiffres significatifs.

Le chargement relit les tailles, crée un réseau de mêmes dimensions et remplit ses paramètres. Un fichier dont l'en-tête, la version ou le nombre de valeurs est incorrect est refusé. Le programme de test entraîne le réseau, le sauvegarde, le recharge et compare les deux : leurs sorties sont identiques au bit près.

Choix et justifications

Choix	**Justification**
Une seule couche cachée	Conseillé par le sujet. Une couche cachée est nécessaire, car $\bar{A}.\bar{B} + A.B$ n'est pas linéairement séparable : aucune droite ne sépare les cas qui valent 1 de ceux qui valent 0. Elle suffit pour reconnaître des lettres isolées.

Tailles des couches passées en paramètre	Le même code sert à la preuve de concept (2 → 4 → 2) et servira au réseau final (784 → n → 26 pour les images 28 × 28 de la segmentation).
Structure NeuralNetwork qui regroupe les tailles, les paramètres et les activations	Toutes les fonctions reçoivent un seul pointeur. Les activations restent dans la structure après la propagation avant et sont réutilisées par la rétropropagation, sans recalcul.
Poids rangés dans des tableaux à une dimension (W[i × input_size + j])	Chaque couche tient dans un seul bloc mémoire : une seule allocation, un parcours dans l'ordre de la mémoire, et une sauvegarde qui écrit le tableau tel quel.
Structure Gradient séparée, de même forme que les paramètres	Le calcul du gradient est séparé de la mise à jour des poids. Cela permettra de cumuler les gradients de plusieurs exemples pour passer aux mini-lots avec les lettres.
Structure Dataset de paires (entrée, sortie attendue)	L'entraînement ne dépend pas du problème : la même fonction reçoit les 4 cas logiques aujourd'hui, et recevra les images de lettres plus tard.
Sigmoïde en couche cachée, softmax en sortie	Conseillé par le sujet. Les sorties sont des probabilités, et la classe reconnue est la plus probable.
Deux sorties plutôt qu'une	Le softmax d'une seule valeur vaut toujours 1 : il faut au moins deux sorties. Deux sorties codées en one-hot reprennent le schéma des 26 lettres.
Entropie croisée	Associée au softmax, elle donne une erreur de sortie très simple (y − t). Elle évite le ralentissement de l'apprentissage du coût quadratique quand les neurones saturent.
Initialisation de Xavier, uniforme sur ±√(6 / (n_in + n_out))	Avec des poids tous nuls, les neurones cachés resteraient identiques. Avec des poids trop grands, les sigmoïdes saturent dès le départ.
Sauvegarde dans un fichier texte versionné, valeurs écrites avec %.17g	Le fichier se lit à l'œil pour déboguer, et 17 chiffres suffisent pour relire exactement un double. La version permettra de changer de format sans charger un ancien fichier par erreur.

Difficultés

- **Le réseau n'apprenait pas avec une seule sortie.** La première version passait une sortie unique au softmax, qui valait donc toujours 1 : la perte ne pouvait pas baisser. Je l'ai remplacée par deux sorties codées en one-hot.
- **Écriture hors du tableau à l'initialisation.** La boucle qui initialise les poids de sortie parcourait hidden × input cases au lieu de hidden × output, et écrivait donc en dehors du tableau. J'ai ajouté AddressSanitizer à la compilation dès le 3 octobre pour repérer ce type

d’erreur.

- **log(0) dans l’entropie croisée.** Une probabilité nulle donnait une perte infinie. La probabilité est maintenant bornée à 10^{-15} avant le calcul du logarithme.
- **Stabilité numérique du softmax.** Pour de grandes valeurs de z, e^z dépasse la capacité d’un double. Soustraire le maximum avant le calcul ne change pas le résultat et supprime le problème.
- **Libération de la mémoire en cas d’erreur.** Si une allocation échouait au milieu de la création du réseau, la structure elle-même n’était pas libérée. Chaque chemin d’erreur libère maintenant tout ce qui a déjà été alloué.
- **Intégration au dépôt commun.** Deux ajustements ont été nécessaires :
    - les fonctions de gradient ont été déplacées dans leur propre fichier ;
    - le dossier de sauvegarde n’existait pas dans un dépôt fraîchement cloné, car Git ne suit pas les dossiers vides : un fichier .gitkeep le crée

Solver (Antoine Muh)

Le programme solver charge une grille depuis un fichier texte, cherche un mot dans les 8 directions et affiche les coordonnées de sa première et de sa dernière lettre, au format (x0,y0)(x1,y1), ou Not found si le mot est absent. Il passe les 8 exemples du sujet : par exemple, ./solver grid HORIZONTAL affiche (0,0)(9,0).

Algorithme

Pour chaque case qui contient la première lettre du mot, le solver essaie les 8 directions. Une direction est un déplacement (dx, dy) lu dans deux tableaux de 8 valeurs. Le solver avance lettre par lettre, vérifie que la case est dans la grille avant de la lire et s'arrête à la première lettre différente. Au premier mot complet, la dernière lettre est en (x + dx × (n − 1), y + dy × (n − 1)), où n est la longueur du mot.

Choix et justifications

Choix	**Justification**
Grille en tableau de chaînes (char **), agrandi avec realloc	Accès direct à grid[y][x], aucune limite de taille ; format simple à remplir plus tard avec les lettres reconnues par le réseau.
Lecture avec getline	Alloue une ligne de la bonne longueur, sans tampon de taille fixe.
Grille irrégulière rejetée au chargement	Évite que la recherche lise en dehors d'une ligne.
Directions dans deux tableaux plutôt que 8 cas écrits à la main	Une seule boucle couvre les 8 directions : moins de code et moins d'occasions d'erreur.
Recherche directe depuis chaque case	Les grilles font une dizaine de cases de côté : la recherche est instantanée, une structure plus complexe n'apporterait rien.
Mot mis en majuscules	Les exemples du sujet passent des mots en minuscules sur une grille en majuscules.

La fonction de recherche n'affiche rien et renvoie le résultat	La même fonction sert au programme solver, aux tests et au programme final.

Tests

Un programme de test en C, compilé par make solver_tests, appelle directement la fonction de recherche sur les 8 exemples du sujet. Pour chaque mot, il affiche le résultat attendu et le résultat obtenu. Ce fichier n'a pas été ajouté au repo.

Difficultés

- Gestion de la mémoire : fuites sur les chemins d'erreur du chargement et lignes comptées deux fois. Corrigées en libérant tout à chaque sortie de la fonction, puis vérifiées avec AddressSanitizer.
- Coordonnées de fin fausses à cause de la priorité des opérateurs : le calcul a été réécrit avec des parenthèses explicites.
- Fonction getline invisible avec -std=c11 : c'est une fonction POSIX, rendue visible par #define _GNU_SOURCE.
- Intégration au dépôt : conflit Git sur le Makefile commun, résolu en gardant les quelques modifications, et fichier de test supprimé.

Tâches futures pour la suite du projet

Gustave

Ma première tâche principale sera de mettre en place l'algorithme de rotation automatique, indispensable au bon découpage de l'image pour le réseau de neurones. Ensuite, j'ajouterai les fonctions d'affichage de la grille terminée dans la zone de prévisualisation, et implémenterai les derniers boutons permettant de passer de la grille avant analyse à la grille terminée. Une fois ces tâches terminées, je retravaillerai éventuellement les algorithmes de prétraitement pour les optimiser et minimiser les erreurs d'analyse.

Guillaume

Pour la soutenance finale, ma mission est de passer de la preuve de concept au réseau qui reconnaît les lettres. J'adapterai d'abord le réseau aux images 28 × 28 fournies par la segmentation : 784 entrées, une couche cachée dont je fixerai la taille par des essais, et 26 sorties, une par lettre. J'écrirai une fonction qui reçoit l'image d'une lettre, la convertit en valeurs entre 0 et 1 et renvoie la lettre reconnue. Elle servira directement à la reconstruction de la grille et de la liste de mots. J'entraînerai ensuite le réseau sur le jeu de données de caractères généré à partir de polices. J'y ajouterai des variations (légère rotation, bruit, décalage, épaisseur du trait) pour qu'il reste fiable sur les lettres réellement découpées dans les images. Pour accélérer et stabiliser l'apprentissage, je mélangerai les exemples à chaque époque et je passerai à un entraînement par mini-lots. Je mesurerai le taux de reconnaissance sur des exemples non utilisés pendant l'apprentissage, puis sur les lettres extraites des images du sujet. Les poids entraînés seront sauvegardés avec le format déjà en place, et le programme final les chargera au démarrage sans avoir à réentraîner le réseau.

Antoine Lapp

Pour la suite, je vais me charger d'appeler le réseau de neurones de Guillaume sur chaque caractère détecté et ainsi créer un fichier texte contenant la grille et une liste de chaînes de caractères qui correspondra aux mots à trouver. On pourra donc appeler le solver d'Antoine M et ainsi résoudre la grille. Je vais sûrement également aider Gustave à afficher la grille de résultat en traçant des rectangles sur les mots détectés. Enfin, ma tâche principale sera d'améliorer l'analyse de l'image, notamment la classification des régions pour que la résolution fonctionne sur des images plus complexes.

Antoine Muh

Pour la soutenance finale, ma mission porte sur la robustesse de la segmentation et sur sa liaison avec le réseau de neurones. Je rendrai d'abord la segmentation capable de traiter les images de niveau 3, où la grille n'a pas de bordures, puis les listes de mots disposées sur plusieurs colonnes. Je fiabiliserai ensuite la segmentation sur les images de niveaux 2 et 3, pour que les lettres soient découpées correctement sur toutes les images du sujet. Je relierai aussi la segmentation au réseau de neurones, pour que les lettres découpées lui soient transmises directement au format attendu. Pour l'entraînement du réseau, je générerai un jeu de données de caractères à partir de polices. Enfin, je mettrai en place une intégration continue qui vérifiera, à chaque envoi sur le dépôt, que le projet compile sans avertissement et sans erreur mémoire.

Bibliographie

Commune

- David Bouchet, OCR Word Search Solver, sujet du projet, EPITA S3, 2026/2027.
- ISO/IEC 9899:2011, Programming languages, C (norme C11).
- GNU Make Manual : https://www.gnu.org/software/make/manual/make.html
- AddressSanitizer : https://github.com/google/sanitizers/wiki/AddressSanitizer
- Scott Chacon et Ben Straub, Pro Git, 2e édition : https://git-scm.com/book/fr/v2
- Documentation de SDL2 : https://wiki.libsdl.org/SDL2/FrontPage

Gustave

- Comment afficher du texte avec SDL2 : https://youtu.be/JX4Wvi6E8jQ?si=4pxst7qKqY_4uknl
- Étude et algorithme de Derek Bradley et Gerhard Roth sur la binarisation à seuil adaptatif : https://people.scs.carleton.ca/~roth/iit-publications-iti/docs/gerh-50002.pdf
- Site utilisé pour la conversion de texte en image transparente : https://www.photo-pick.com/fr/utils/text-to-png
- Informations sur les filtres médians pour débruitage d'image : https://evidentscientific.com/en/microscope-resource/tutorials/digital-imaging/processing/medianfilter
- Algorithme de filtre médian pour débruitage d'image : https://www.f-legrand.fr/scidoc/docmml/numerique/filtre/median/median.html
- EPITA, TP de programmation "Howl's Machine" sur les bases de SDL

Guillaume

- Michael Nielsen, *Neural Networks and Deep Learning*, Determination Press, 2015 : http://neuralnetworksanddeeplearning.com/
- Ian Goodfellow, Yoshua Bengio et Aaron Courville, *Deep Learning*, MIT Press, 2016, chapitre 6 : https://www.deeplearningbook.org
- Xavier Glorot et Yoshua Bengio, « Understanding the difficulty of training deep feedforward neural networks », AISTATS, 2010.
- GeeksforGeeks, *The Role of Softmax in Neural Networks: Detailed Explanation and Applications* : https://www.geeksforgeeks.org/deep-learning/the-role-of-softmax-in-neural-networks-detailed-explanation-and-applications/
- Wikipédia, *Softmax function* : https://en.wikipedia.org/wiki/Softmax_function
- Pages de manuel Linux rand(3), exp(3), fprintf(3) et fscanf(3) : https://man7.org/linux/man-pages/
- 365 Data Science, *What Is Xavier Initialization?* : https://365datascience.com/tutorials/machine-learning-tutorials/what-is-xavier-initialization/

Antoine Lapp

- Jaekyu Ha & Robert M. Haralick Ihsin T. Phillips, *Recursive X-Y Cut using Bounding Boxes of Connected Components* : https://www.haralick.org/conferences/71280952.pdf
- F. Cesarini M. Gori S. Marinai G. Soda, *Structured Document Segmentation and Representation by the Modified X-Y tree*, Università degli studi firenze, 2026 : https://flore.unifi.it/bitstream/2158/597411/1/ICDAR99a.pdf
- EPITA, cours d'algorithmique sur les arbres généraux
- Lifeng He Xiwei Ren Qihang Gao Xiao Zhao Bin Yao Yuyan Chao, *The connected-component labeling problem: A review of state-of-the-art algorithms* : https://www.sciencedirect.com/science/article/pii/S0031320317301693?via%3Dihub
- Pages de manuel Linux de math.h

Antoine MUH

- Pages de manuel Linux getline(3), realloc(3), toupper(3) et feature_test_macros(7) : https://man7.org/linux/man-pages/